Python Activity-Based Learning

Case Study 29: Password Policy Validation Engine

1. Problem Statement

Check sample password strings against simple password rules and report every rule that fails.

2. Objectives

- Use string operations to inspect password characters.
- Apply multiple conditional checks.
- Report all failed rules instead of stopping at the first failure.

3. Assumptions

- A valid password has at least 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character from !@#$%.
- The password must not contain the prohibited substring 'password' in any letter case.
- This is a basic classroom validation exercise, not a complete real-world security checker.

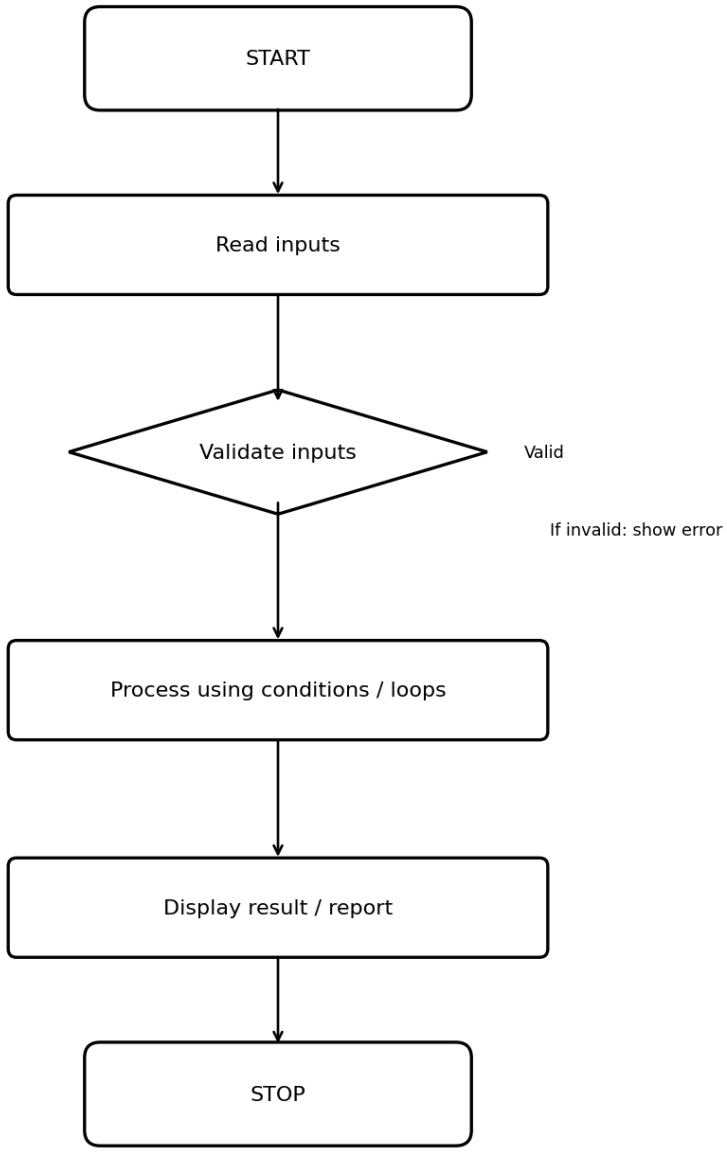
4. Algorithm

1. Read a password string.
2. Create an empty list of failed rules.
3. Check length, uppercase, lowercase, digit, special character, and prohibited substring.
4. Add a message for each failed rule.
5. If the list is empty, report valid; otherwise print each failed rule.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: Password Policy Validation Engine



6. Pseudocode

```
START
READ password
CREATE empty list errors
IF length < 8 THEN add length error
IF no uppercase character THEN add uppercase error
IF no lowercase character THEN add lowercase error
IF no digit THEN add digit error
IF no allowed special character THEN add special-character error
IF lowercase password contains 'password' THEN add prohibited-word error
IF errors is empty THEN DISPLAY valid
ELSE DISPLAY each error
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
password = input("Enter a sample password: ")
errors = []

if len(password) < 8:
    errors.append("Use at least 8 characters.")
if not any(ch.isupper() for ch in password):
    errors.append("Include at least one uppercase letter.")
if not any(ch.islower() for ch in password):
    errors.append("Include at least one lowercase letter.")
if not any(ch.isdigit() for ch in password):
    errors.append("Include at least one digit.")
if not any(ch in "!@#$%" for ch in password):
    errors.append("Include at least one special character from !@#$%.")
if "password" in password.lower():
    errors.append("Do not include the word 'password'.")

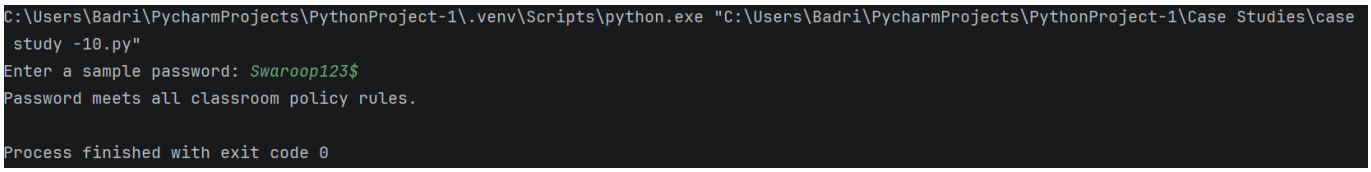
if len(errors) == 0:
    print("Password meets all classroom policy rules.")
else:
    print("Password does not meet the policy:")
    for error in errors:
        print("-", error)
```

8. Test Cases

Test input / condition	Expected result	Purpose
Abcdef1!	8 characters; all categories	Valid
hello	Lowercase only; short	Several failed rules listed
Password1!	Contains prohibited word	Prohibited-word rule fails

9. Sample Output Screenshot

Sample console output for one test run:



10. Conclusion

The program practices string methods, loops, conditions, and list-based error reporting. It is intended for learning and not as a production authentication system.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.